

빅데이터 개론

기말 프로젝트 보고서



수업명	빅데이터 개론
프로젝트명	와인 품질 분류 프로젝트
조	101조
학번	20195171 / 20215196
이름	박준영 / 윤서은

1. 데이터 소개

평소 와인에 관심이 많아 즐겨 마시는데 프로젝트 진행을 위한 데이터를 찾던 중 와인과 관련된 데이터셋을 발견하였습니다. 와인을 마시면서 들었던 의문은 “**좋은 와인이란 무엇인가, 와인의 품질을 결정하는 것은 무엇인가?**”에 대한 의문을 해결해주는 모델을 만들어보고 싶어 해당 데이터셋을 선택하게 되었습니다.

저희가 선택한 데이터의 원본은 UCI의 와인 품질 데이터입니다. 해당 데이터는 레드 와인과 화이트 와인이 분리되어 있어 캐글에서 해당 데이터들이 하나의 파일로 합쳐진 데이터¹를 가져왔습니다. 이 데이터는 포르투갈의 미뉴 지역에서 생산되는 비뉴 베르드의 레드 와인과 화이트 와인에 대한 데이터입니다.

데이터의 컬럼은 type, fixed acidity, volatile acidity, citric acid, residual sugar, chlorides, free sulfur dioxide, total sulfur dioxide, density, pH, sulphates, alcohol, quality의 총 13개로 구성되어 있습니다. 각 변수에 대한 설명은 아래와 같습니다.

변수명		내용
type of wine	와인 종류	와인 종류 (분류 : '레드', '화이트')
fixed acidity	고정산도	와인을 발효하는 데 사용된 포도에서 자연적으로 발생하여 와인으로 옮겨지는 산. (g / dm^3) 주로 와인을 발효하는 데 사용된 포도에서 유래하는 타르타르산, 사과산, 구연산 또는 석신산으로 구성됨. 또한 쉽게 증발되지 않음.
volatile acidity	휘발성 산도	낮은 온도에서 증발하는 산. (g / dm^3) 주로 아세트산으로, 매우 높은 수준에서 불쾌한 식초와 같은 맛을 낼 수 있음
citric acid	구연산	구연산은 와인의 산도를 높이는 산 보충제로 사용됨. (g / dm^3) 일반적으로 소량으로 발견되며 와인에 '신선함'과 풍미를 더할 수 있습니다.
residual sugar	잔류당	발효가 멈춘 후 남은 당의 양. (g / dm^3) 1g/L 미만인 와인을 찾는 것은 드물. 잔류당 수치가 45g/L 이상인 와인은 달콤한 것으로 간주됨 반면에 달콤한 맛이 나지 않는 와인은 드라이한 것으로 간주됨
chlorides	염화물	와인에 존재하는 염화물 염(염화나트륨)의 양. (g / dm^3)
free sulfur dioxide	자유 이산화황	자유 형태의 SO_2 는 분자 SO_2 (용해 가스)와 중아황산염 이온 사이에 평형 상태로 존재하며, 미생물의 성장과 와인의 산화를 방지. 다른 모든 것이 일정하다면, 자유 이산화황 함량이 높을수록 보존 효과가 더 강함. (mg / dm^3)
total sulfur dioxide	총 이산화황	SO_2 의 자유 형태와 결합 형태의 양 (mg/dm^3) 낮은 농도에서 SO_2 는 대부분 와인에서 감지되지 않지만, 자유 SO_2 농도가 50ppm을 초과하면 SO_2 가 와인의 향과 맛에서 분명해짐.
density	밀도	와인 주스의 밀도는 알코올 함량과 설탕 함량에 따라 달라짐. (g / cm^3) 일반적으로 물의 밀도와 비슷하지만 더 높음. (와인은 '더 진하다').
pH	pH	와인의 산도를 측정하는 척도 대부분 와인은 pH 척도에서 3-4 사이임. pH가 낮을수록 와인의 산성도가 더 높고, pH가 높을수록 와인의 산성도가 낮음. (pH 척도는 기술적으로 와인에 떠다니는 자유 수소 이온의 농도를 측정하는 대수 척도) (pH 척도의 각 지점은 10의 인수임. 즉, pH가 3인 와인은 pH가 4인 와인보다 10배 더 산성입니다.)
sulphates	황산염	평균 및 황산화제로 작용함. (g/dm^3) 와인 첨가물로서 황산칼륨의 양은 이산화황 가스(SO_2) 수치에 영향을 줄 수 있음.
alcohol	알코올	주어진 와인 부피에 얼마나 많은 알코올이 포함되어 있는가(ABV). 와인은 일반적으로 5~15%의 알코올을 포함. (부피 기준 %)
quality	품질	와인 전문가가 매긴 사이의 점수 (0(매우 나쁨)에서 10(매우 우수))

¹ Wine Quality Data Set

<https://www.kaggle.com/datasets/ruthgn/wine-quality-data-set-red-white-wine/data>

2. 데이터의 취득과 정제

데이터 파일(.csv)을 구글드라이브에 업로드 후 공유하여 구글 Colab에서 링크를 통해 데이터를 불러왔습니다.

```
[ ] #https://drive.google.com/file/d/1lZWfTL4Hyc_PZjScJQZwq1ssGLXd3N6q/view?usp=drive_link

system("gdown --id 1lZWfTL4Hyc_PZjScJQZwq1ssGLXd3N6q")
system("ls", TRUE)
```

↗ 'sample_data' · 'wine_quality.csv'

data.table 패키지의 fread() 함수로 데이터를 불러와 wine_quality에 tibble 형식으로 파일을 읽어왔습니다.

```
[ ] wine_quality <- fread("/content/wine_quality.csv", header = T, encoding = "UTF-8") %>% as_tibble
wine_quality %>% show()
```

↗ # A tibble: 6,497 × 13

	type	fixed acidity	volatile acidity	citric acid	residual sugar
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1	white	7	0.27	0.36	20.7
2	white	6.3	0.3	0.34	1.6
3	white	8.1	0.28	0.4	6.9
4	white	7.2	0.23	0.32	8.5
5	white	7.2	0.23	0.32	8.5
6	white	8.1	0.28	0.4	6.9
7	white	6.2	0.32	0.16	7
8	white	7	0.27	0.36	20.7
9	white	6.3	0.3	0.34	1.6
10	white	8.1	0.22	0.43	1.5

6,487 more rows
 # 8 more variables: chlorides <dbl>, free sulfur dioxide <dbl>,
 # total sulfur dioxide <dbl>, density <dbl>, pH <dbl>, sulphates <dbl>,
 # alcohol <dbl>, quality <int>

다음으로는 결측값 처리를 수행하였습니다. 수행한 결과 결측값은 존재하지 않는 것으로 확인되었습니다.

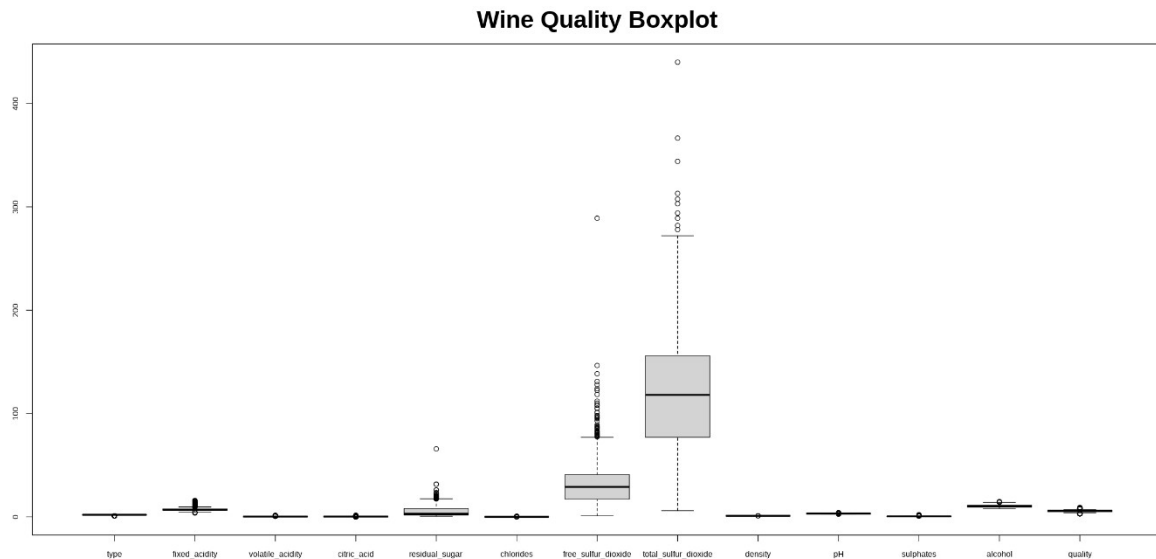
```
[ ] # 결측값 확인
wine_na <- sum(is.na(wine_quality))
wine_na %>% show()

# 결측값 존재하지 않음
```

↗ [1] 0

변수들의 boxplot을 실행하면 아래와 같은 결과를 얻을 수 있습니다.

저희는 모든 변수들로 모델링을 진행하지 않고 품질도와의 상관관계 분석과 중요도 분석을 통해 설명변수를 선택한 후 모델링을 진행하였습니다. 따라서 이상값 처리는 변수 중요도 분석 후 진행하기로 결정하였습니다. 이상값에 대한 처리는 뒤쪽에서 설명 드리겠습니다.



결측값 확인 후 summary()함수를 실행한 결과입니다.

```
summary(wine_quality)
```

type	fixed acidity	volatile acidity	citric acid
Length:6497	Min. : 3.800	Min. : 0.0800	Min. : 0.0000
Class :character	1st Qu.: 6.400	1st Qu.: 0.2300	1st Qu.: 0.2500
Mode :character	Median : 7.000	Median : 0.2900	Median : 0.3100
	Mean : 7.215	Mean : 0.3397	Mean : 0.3186
	3rd Qu.: 7.700	3rd Qu.: 0.4000	3rd Qu.: 0.3900
	Max. : 15.900	Max. : 1.5800	Max. : 1.6600

residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide
Min. : 0.600	Min. : 0.00900	Min. : 1.00	Min. : 6.0
1st Qu.: 1.800	1st Qu.: 0.03800	1st Qu.: 17.00	1st Qu.: 77.0
Median : 3.000	Median : 0.04700	Median : 29.00	Median : 118.0
Mean : 5.443	Mean : 0.05603	Mean : 30.53	Mean : 115.7
3rd Qu.: 8.100	3rd Qu.: 0.06500	3rd Qu.: 41.00	3rd Qu.: 156.0
Max. : 65.800	Max. : 0.61100	Max. : 289.00	Max. : 440.0

density	pH	sulphates	alcohol
Min. : 0.9871	Min. : 2.720	Min. : 0.2200	Min. : 8.00
1st Qu.: 0.9923	1st Qu.: 3.110	1st Qu.: 0.4300	1st Qu.: 9.50
Median : 0.9949	Median : 3.210	Median : 0.5100	Median : 10.30
Mean : 0.9947	Mean : 3.219	Mean : 0.5313	Mean : 10.49
3rd Qu.: 0.9970	3rd Qu.: 3.320	3rd Qu.: 0.6000	3rd Qu.: 11.30
Max. : 1.0390	Max. : 4.010	Max. : 2.0000	Max. : 14.90

quality
Min. : 3.000
1st Qu.: 5.000
Median : 6.000
Mean : 5.818
3rd Qu.: 6.000
Max. : 9.000

3. 데이터 가공

우선 제일 처음으로 진행한 과정은 데이터의 자료형을 확인한 것입니다. `str()` 함수를 이용해 진행한 결과 `type` 변수는 `char`타입, `quality` 변수는 `int`타입 그리고 나머지 변수들은 `num` 타입인 것을 확인할 수 있었습니다.

```
# 와인 품질 데이터 자료형 확인
```

```
wine_quality %>% str()
cat("\n")
```

```
# type은 char
```

```
# quality는 int
```

```
# 나머지 변수들은 num
```

```
tibble [6,497 × 13] (S3: tbl_df/tbl/data.frame)
```

```
$ type           : chr [1:6497] "white" "white" "white" "white" ...
$ fixed acidity  : num [1:6497] 7 6.3 8.1 7.2 7.2 8.1 6.2 7 6.3 8.1 ...
$ volatile acidity : num [1:6497] 0.27 0.3 0.28 0.23 0.23 0.28 0.32 0.27 0.3 0.22 ...
$ citric acid    : num [1:6497] 0.36 0.34 0.4 0.32 0.32 0.4 0.16 0.36 0.34 0.43 ...
$ residual sugar : num [1:6497] 20.7 1.6 6.9 8.5 8.5 6.9 7 20.7 1.6 1.5 ...
$ chlorides      : num [1:6497] 0.045 0.049 0.05 0.058 0.058 0.05 0.045 0.045 0.044 ...
$ free sulfur dioxide : num [1:6497] 45 14 30 47 47 30 30 45 14 28 ...
$ total sulfur dioxide: num [1:6497] 170 132 97 186 186 97 136 170 132 129 ...
$ density        : num [1:6497] 1.001 0.994 0.995 0.996 0.996 ...
$ pH             : num [1:6497] 3 3.3 3.26 3.19 3.19 3.26 3.18 3 3.3 3.22 ...
$ sulphates      : num [1:6497] 0.45 0.49 0.44 0.4 0.4 0.44 0.47 0.45 0.49 0.45 ...
$ alcohol        : num [1:6497] 8.8 9.5 10.1 9.9 9.9 10.1 9.6 8.8 9.5 11 ...
$ quality        : int [1:6497] 6 6 6 6 6 6 6 6 6 6 ...
- attr(*, ".internal.selfref")=<externalptr>
```

4. 데이터 시각화

데이터 가공 과정을 수행하며 시각화 과정을 동시에 진행하였습니다.

우선 처음으로 red와 white로 이루어진 type 변수에 대한 factor 변환을 진행하였습니다.

```
[ ] # type factor 변환
wine_quality$type <- as.factor(wine_quality$type)

str(wine_quality$type)
```

➡ Factor w/ 2 levels "red","white": 2 2 2 2 2 2 2 2 2 2 ...

그 다음으로는 데이터 전처리를 진행하기 위한 데이터의 지표를 확인했습니다.

```
# 데이터 지표 확인
wine_quality %>% select_if(is.numeric) %>% describe() %>% round(2)
```

➡

A psych: 12 × 13													
	vars	n	mean	sd	median	trimmed	mad	min	max	range	skew	kurtosis	se
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
fixed acidity	1	6497	7.22	1.30	7.00	7.06	0.89	3.80	15.90	12.10	1.72	5.05	0.02
volatile acidity	2	6497	0.34	0.16	0.29	0.32	0.12	0.08	1.58	1.50	1.49	2.82	0.00
citric acid	3	6497	0.32	0.15	0.31	0.32	0.10	0.00	1.66	1.66	0.47	2.39	0.00
residual sugar	4	6497	5.44	4.76	3.00	4.70	2.52	0.60	65.80	65.20	1.43	4.35	0.06
chlorides	5	6497	0.06	0.04	0.05	0.05	0.02	0.01	0.61	0.60	5.40	50.84	0.00
free sulfur dioxide	6	6497	30.53	17.75	29.00	29.32	17.79	1.00	289.00	288.00	1.22	7.90	0.22
total sulfur dioxide	7	6497	115.74	56.52	118.00	115.92	57.82	6.00	440.00	434.00	0.00	-0.37	0.70
density	8	6497	0.99	0.00	0.99	0.99	0.00	0.99	1.04	0.05	0.50	6.60	0.00
pH	9	6497	3.22	0.16	3.21	3.21	0.16	2.72	4.01	1.29	0.39	0.37	0.00
sulphates	10	6497	0.53	0.15	0.51	0.52	0.12	0.22	2.00	1.78	1.80	8.64	0.00
alcohol	11	6497	10.49	1.19	10.30	10.40	1.33	8.00	14.90	6.90	0.57	-0.53	0.01
quality	12	6497	5.82	0.87	6.00	5.79	1.48	3.00	9.00	6.00	0.19	0.23	0.01

변수들 중 quality 변수는 연속성 데이터이므로 분석 성능을 높이기 위해 0 ~ 5는 기본 품질(0)로 6 ~ 10은 우수품질(1)로 설정해 범주형 데이터로 변환하였습니다. 먼저 wine_quality 데이터에서 quality 부분만 선택해 new_wine_quality라는 변수에 저장했습니다.

```
# wine_quality에서 quality 부분만 선택 new_wine_quality
new_wine_quality <- select(wine_quality, quality)
new_wine_quality %>% show()

# A tibble: 6,497 × 1
  quality
  <int>
1       6
2       6
3       6
4       6
5       6
6       6
7       6
8       6
9       6
10      6
# i 6,487 more rows
```

그 다음 위에서 설정한 범위에 따라 기본 품질과 우수 품질로 구분하였습니다.

```
# 0 ~ 5(기본품질) : 0 / 6 ~ 10(우수품질) : 1 로 구분
quality_target <- new_wine_quality %>% mutate(new_wine_quality = ifelse(quality < 6, "기본품질", ifelse (quality >= 6 | quality <= 10, "우수품질")))
quality_target %>% show()

# A tibble: 6,497 × 2
  quality new_wine_quality
  <int> <chr>
1      6 우수품질
2      6 우수품질
3      6 우수품질
4      6 우수품질
5      6 우수품질
6      6 우수품질
7      6 우수품질
8      6 우수품질
9      6 우수품질
10     6 우수품질
# i 6,487 more rows
```

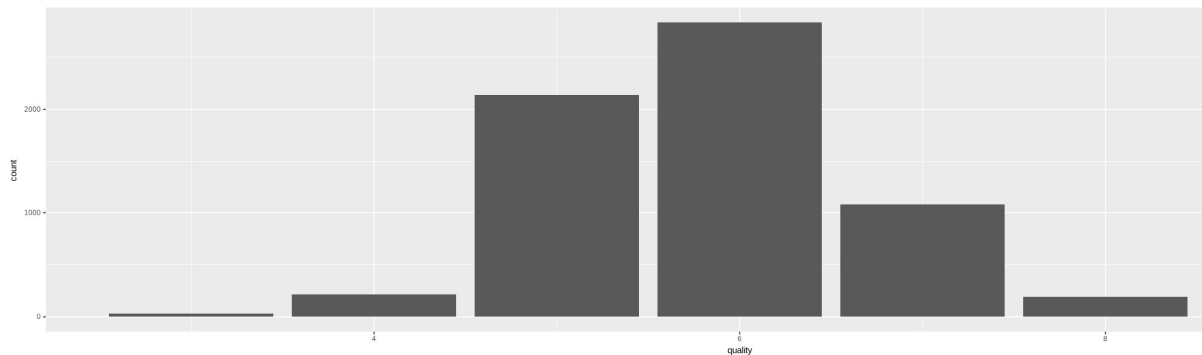
마지막으로 해당 데이터를 새로 설정한 품질도(quality_target)라는 변수에 저장하는 과정을 진행했습니다.

```
# factor로 변환
wine_quality$quality_target <- factor(quality_target$new_wine_quality)
wine_quality
```

A tibble: 6497 × 14

type	fixed_acidity	volatile_acidity	citric_acid	residual_sugar	chlorides	free_sulfur_dioxide	total_sulfur_dioxide	density	pH	sulphates	alcohol	quality	quality_target
<fct>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<int>	<fct>
white	7.0	0.27	0.36	20.70	0.045	45	170	1.0010	3.00	0.45	8.8	6	우수품질
white	6.3	0.30	0.34	1.60	0.049	14	132	0.9940	3.30	0.49	9.5	6	우수품질
white	8.1	0.28	0.40	6.90	0.050	30	97	0.9951	3.26	0.44	10.1	6	우수품질
white	7.2	0.23	0.32	8.50	0.058	47	186	0.9956	3.19	0.40	9.9	6	우수품질
white	7.2	0.23	0.32	8.50	0.058	47	186	0.9956	3.19	0.40	9.9	6	우수품질
white	8.1	0.28	0.40	6.90	0.050	30	97	0.9951	3.26	0.44	10.1	6	우수품질
white	6.2	0.32	0.16	7.00	0.045	30	136	0.9949	3.18	0.47	9.6	6	우수품질
white	7.0	0.27	0.36	20.70	0.045	45	170	1.0010	3.00	0.45	8.8	6	우수품질
white	6.3	0.30	0.34	1.60	0.049	14	132	0.9940	3.30	0.49	9.5	6	우수품질
white	8.1	0.22	0.43	1.50	0.044	28	129	0.9938	3.22	0.45	11.0	6	우수품질
white	8.1	0.27	0.41	1.45	0.033	11	63	0.9908	2.99	0.56	12.0	5	기본품질
white	8.6	0.23	0.40	4.20	0.035	17	109	0.9947	3.14	0.53	9.7	5	기본품질
white	7.9	0.18	0.37	1.20	0.040	16	75	0.9920	3.18	0.63	10.8	5	기본품질
white	6.6	0.16	0.40	1.50	0.044	48	143	0.9912	3.54	0.52	12.4	7	우수품질
white	8.3	0.42	0.62	19.25	0.040	41	172	1.0002	2.98	0.67	9.7	5	기본품질
white	6.6	0.17	0.38	1.50	0.032	28	112	0.9914	3.25	0.55	11.4	7	우수품질
white	6.3	0.48	0.04	1.10	0.046	30	99	0.9928	3.24	0.36	9.6	6	우수품질

품질도 변수의 범위를 0~5, 6~10을 기준으로 나눈 이유는 아래의 데이터 분포를 보면 알 수 있습니다. 품질변수는 4~7의 중간 값에 집중되어 있어 위와 같은 기준으로 데이터를 나누는 것이 합리적이라 판단했기 때문입니다. 또한 데이터 불균형을 고려하여 이와 같이 진행하는 것으로 결정하였습니다.



변수명들이 다 영어로 되어있어 데이터 분석 진행을 더 수월하게 진행하기 위해서 변수명들을 한글로 변경하는 과정을 진행하였습니다.

```
# 열 이름이 모두 영어로 되어있어 한글로 변경
colnames(wine_quality) <- c("와인_종류", "고정_산도", "휘발성_산도", "구연산", "잔류_당", "염화물", "자유_이산화황", "총_이산화황", "밀도", "pH", "황산염", "알코올", "품질", "품질도")
wine_quality
```

A tibble: 6497 × 14

와인_종류	고정_산도	휘발성_산도	구연산	잔류_당	염화물	자유_이산화황	총_이산화황	밀도	pH	황산염	알코올	품질	품질도
<fct>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<int>	<fct>
white	7.0	0.27	0.36	20.70	0.045	45	170	1.0010	3.00	0.45	8.8	6	우수품질
white	6.3	0.30	0.34	1.60	0.049	14	132	0.9940	3.30	0.49	9.5	6	우수품질
white	8.1	0.28	0.40	6.90	0.050	30	97	0.9951	3.26	0.44	10.1	6	우수품질
white	7.2	0.23	0.32	8.50	0.058	47	186	0.9956	3.19	0.40	9.9	6	우수품질
white	7.2	0.23	0.32	8.50	0.058	47	186	0.9956	3.19	0.40	9.9	6	우수품질
white	8.1	0.28	0.40	6.90	0.050	30	97	0.9951	3.26	0.44	10.1	6	우수품질
white	6.2	0.32	0.16	7.00	0.045	30	136	0.9949	3.18	0.47	9.6	6	우수품질
white	7.0	0.27	0.36	20.70	0.045	45	170	1.0010	3.00	0.45	8.8	6	우수품질
white	6.3	0.30	0.34	1.60	0.049	14	132	0.9940	3.30	0.49	9.5	6	우수품질
white	8.1	0.22	0.43	1.50	0.044	28	129	0.9938	3.22	0.45	11.0	6	우수품질
white	8.1	0.27	0.41	1.45	0.033	11	63	0.9908	2.99	0.56	12.0	5	기본품질
white	8.6	0.23	0.40	4.20	0.035	17	109	0.9947	3.14	0.53	9.7	5	기본품질
white	7.9	0.18	0.37	1.20	0.040	16	75	0.9920	3.18	0.63	10.8	5	기본품질
white	6.6	0.16	0.40	1.50	0.044	48	143	0.9912	3.54	0.52	12.4	7	우수품질
white	8.3	0.42	0.62	19.25	0.040	41	172	1.0002	2.98	0.67	9.7	5	기본품질
white	6.6	0.17	0.38	1.50	0.032	28	112	0.9914	3.25	0.55	11.4	7	우수품질

저희는 품질 변수를 이용해 품질도라는 새로운 변수를 만드는 과정을 진행했습니다. 이후 데이터 전처리를 진행할 시 품질 변수를 남겨두면 과적합의 위험성이 있다고 판단해 품질 변수를 삭제하기로 결정하였습니다. 아래는 품질 변수를 삭제한 후의 데이터 요약본입니다.

```
#품질변수를 이용해 품질도라는 새로운 변수를 만들었기에 과적합 방지를 위해 품질 변수 삭제
wine_quality <- wine_quality %>% select(-품질)
```

```
summary(wine_quality)
```

와인_종류	고정_산도	휘발성_산도	구연산
red :1599	Min. : 3.800	Min. :0.0800	Min. :0.0000
white:4898	1st Qu.: 6.400	1st Qu.:0.2300	1st Qu.:0.2500
	Median : 7.000	Median :0.2900	Median :0.3100
	Mean : 7.215	Mean :0.3397	Mean :0.3186
	3rd Qu.: 7.700	3rd Qu.:0.4000	3rd Qu.:0.3900
	Max. :15.900	Max. :1.5800	Max. :1.6600

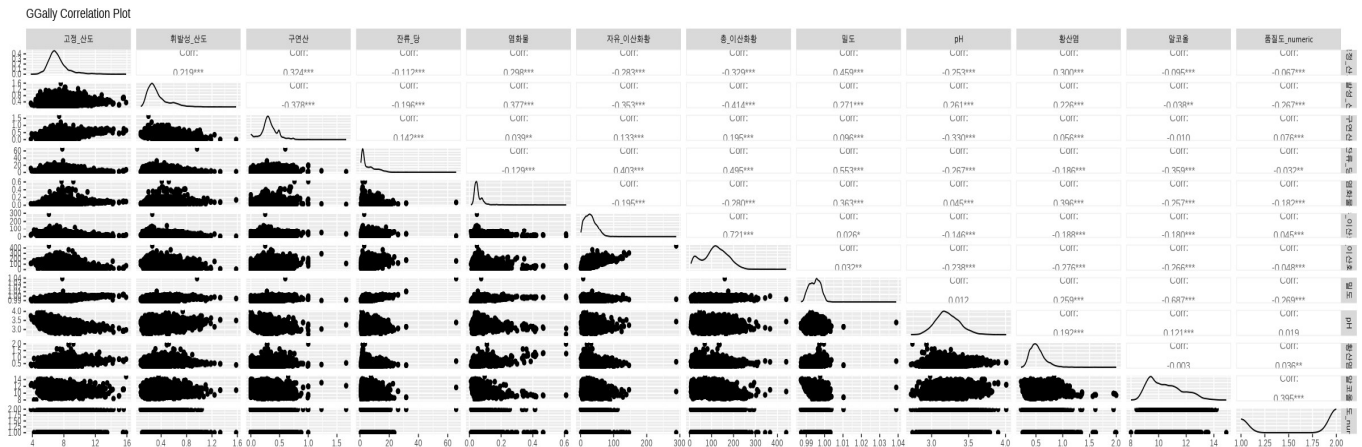
잔류_당	염화물	자유_이산화황	총_이산화황
Min. : 0.600	Min. :0.00900	Min. : 1.00	Min. : 6.0
1st Qu.: 1.800	1st Qu.:0.03800	1st Qu.: 17.00	1st Qu.: 77.0
Median : 3.000	Median :0.04700	Median : 29.00	Median :118.0
Mean : 5.443	Mean :0.05603	Mean : 30.53	Mean :115.7
3rd Qu.: 8.100	3rd Qu.:0.06500	3rd Qu.: 41.00	3rd Qu.:156.0
Max. :65.800	Max. :0.61100	Max. :289.00	Max. :440.0

밀도	pH	황산염	알코올
Min. :0.9871	Min. :2.720	Min. :0.2200	Min. : 8.00
1st Qu.:0.9923	1st Qu.:3.110	1st Qu.:0.4300	1st Qu.: 9.50
Median :0.9949	Median :3.210	Median :0.5100	Median :10.30
Mean :0.9947	Mean :3.219	Mean :0.5313	Mean :10.49
3rd Qu.:0.9970	3rd Qu.:3.320	3rd Qu.:0.6000	3rd Qu.:11.30
Max. :1.0390	Max. :4.010	Max. :2.0000	Max. :14.90

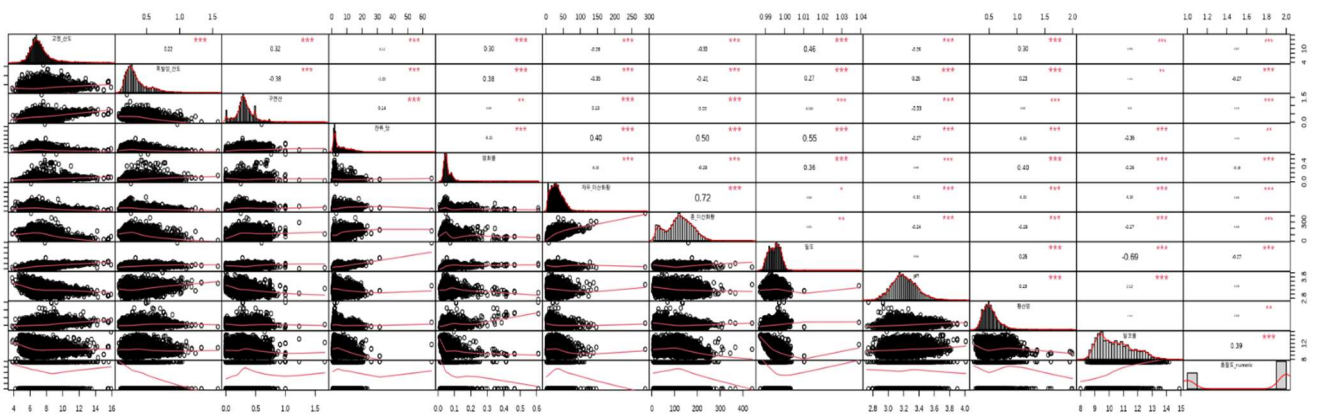
품질도
기본품질:2384
우수품질:4113

설명 변수를 선택하기 위한 데이터 상관계수 분석과 변수 중요도 분석을 진행한 결과는 아래와 같습니다. 저희는 변수 간 관계와 유의성을 평가하기 위해 다양한 기법을 활용해 보았습니다. 우선 상관계수 분석을 위해 numeric타입이 아닌 품질도 변수를 factor 타입에서 numeric 변수로 타입을 변환하였습니다.

첫 번째 그래프는 GGally 패키지의 ggpairs를 활용해 시각화하였습니다.

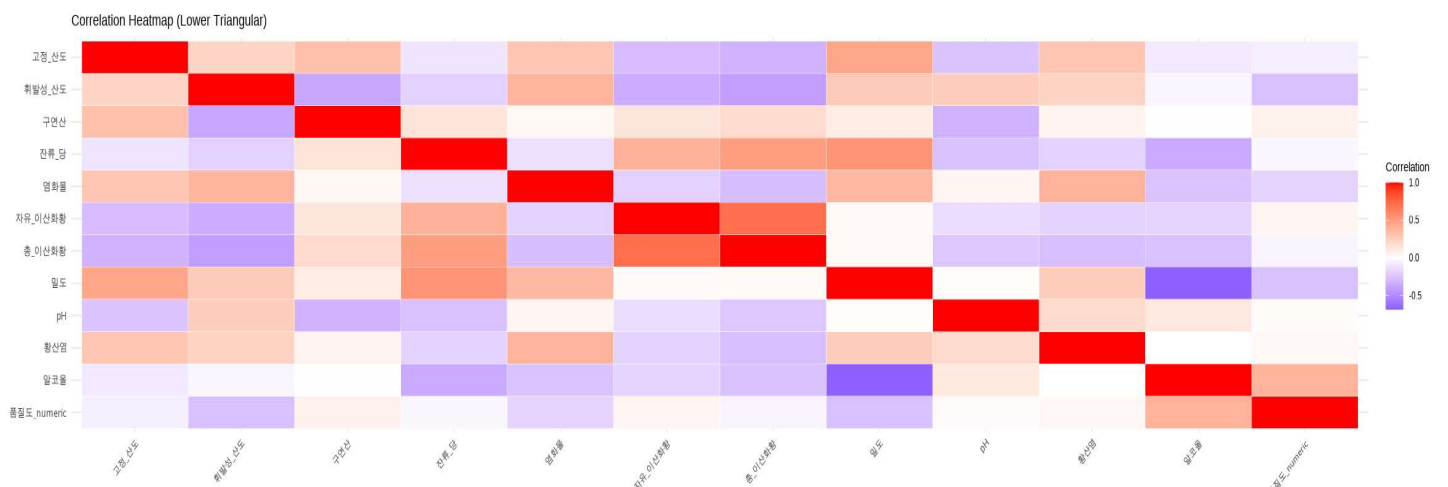


두 번째는 PerformanceAnalytics 패키지를 활용해 생성된 상관관계 플롯입니다. 아래의



그래프를 살펴보면 상관관계수 옆의 *** 표시는 상관관계가 통계적으로 유의미함을 나타내며, 이 정보는 중요한 설명변수를 선정하는데 중요한 역할을 합니다.

세 번째는 상관관계수를 히트맵으로 표현한 그래프입니다. 상관관계가 높을수록 진한 색상을 띄웁니다. 양의 상관관계는 빨간색을, 음의 상관관계는 파란색을 띄웁니다.



마지막으로 진행한 분석은 ANOVA 분석입니다. 이 결과 알코올, 밀도, 휘발성 산도, 염화물, 구연산의 5개의 변수가 품질도와 상관관계가 높은 것을 알 수 있습니다.

```
# ANOVA 분석
anova_results <- list()
for (var in setdiff(numeric_vars, "품질도_numeric")) {
  aov_result <- aov(as.formula(paste(var, "~ 품질도")), data = wine_quality)
  anova_results[[var]] <- summary(aov_result)
}

# ANOVA 결과에서 유의한 변수 선택
anova_p_values <- sapply(anova_results, function(x) x[[1]]$Pr(>F))[1]
anova_significant_vars <- names(sort(anova_p_values))[1:5]
print(anova_significant_vars)
```

[1] "알코올" "밀도" "휘발성_산도" "염화물" "구연산"

위와 같이 변수의 상관관계를 분석한 후 변수 중요도 분석을 진행했습니다. CART 알고리즘 기반의 rpart 함수를 활용해 변수의 중요도를 분석했습니다. 변수 중요도 분석 결과 알코올, 밀도, 휘발성 산도, 염화물, 잔류당이 상위 변수로 설정되었습니다.

```
[ ] # CART 모델로 변수 중요도 확인
library(rpart)

# 품질도를 반응변수로 하는 분류 모델 생성
cart_model <- rpart(품질도 ~ ., data = wine_quality %>% select(-품질도_numeric))

# 변수 중요도 추출
importance <- as.data.frame(cart_model$variable.importance)
importance <- importance[order(importance$`cart_model$variable.importance`, decreasing = TRUE), , drop = FALSE]
colnames(importance) <- "Importance"
print(importance)

# 상위 5개 변수 선택
top_5_vars <- rownames(importance)[1:5]
print(top_5_vars)
```

```
Importance
알코올      406.247308
밀도        194.516790
휘발성_산도 144.511099
염화물      114.529048
잔류_당     108.512910
총_이산화황 100.341458
자유_이산화황 60.918516
pH          3.853629
[1] "알코올" "밀도" "휘발성_산도" "염화물" "잔류_당"
```

상관계수와 변수 중요도 분석을 기반으로 상위 5개의 변수를 이용해 설명 변수를 선택 하였습니다. 저희가 최종적으로 선택한 변수들은 **"알코올, 밀도, 휘발성 산도, 구연산, 잔류당"** 변수입니다. 채택 과정에서 염화물을 제외한 후 위의 5개 변수를 선택하였습니다. **염화물 변수를 제외한 이유**는 해당 성분이 와인의 짠맛을 결정하는 성분이지만, 와인에는 미량만 함유되어 있어 와인 품질 영향에 미칠 가능성이 낮다고 판단했기 때문입니다. 이렇게 선정된 5가지의 설명변수와 품질도 변수만 남기는 과정을 진행했습니다.

```
# 상관계수 기반 상위 변수와 CART 변수 중요도 기반 상위 변수를 통합
top_vars_combined <- unique(c(anova_significant_vars, top_5_vars))
print(top_vars_combined)
```

[1] "알코올" "밀도" "휘발성_산도" "염화물" "구연산"
[6] "잔류_당"

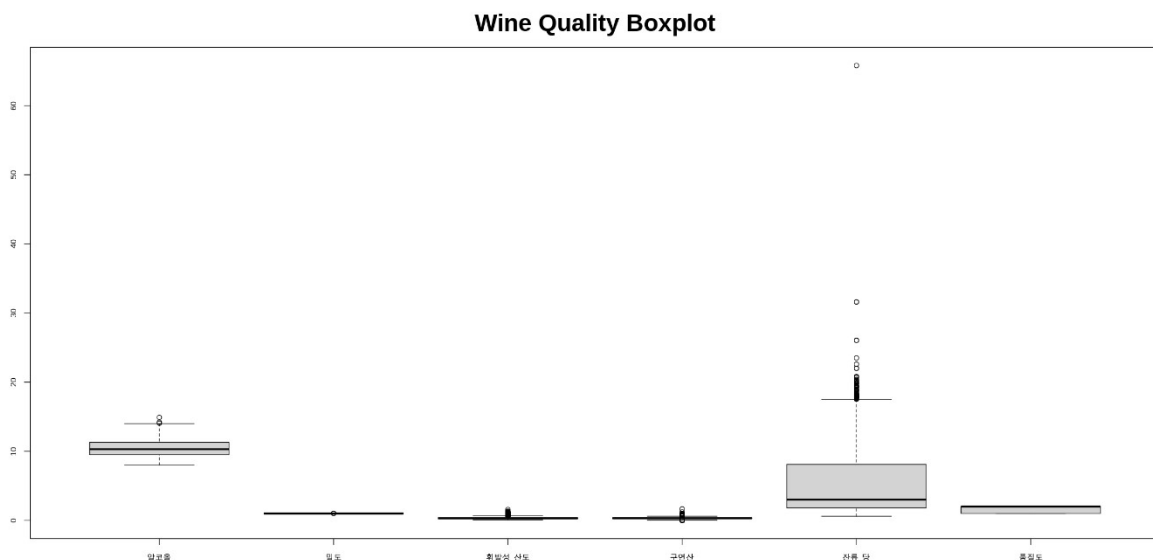
```
# 필요한 변수만 남기기
wine_quality <- wine_quality %>%
  select(알코올, 밀도, 휘발성_산도, 구연산, 잔류_당, 품질도)

summary(wine_quality)
```

알코올	밀도	휘발성_산도	구연산
Min. : 8.00	Min. : 0.9871	Min. : 0.0800	Min. : 0.0000
1st Qu.: 9.50	1st Qu.: 0.9923	1st Qu.: 0.2300	1st Qu.: 0.2500
Median : 10.30	Median : 0.9949	Median : 0.2900	Median : 0.3100
Mean : 10.49	Mean : 0.9947	Mean : 0.3397	Mean : 0.3186
3rd Qu.: 11.30	3rd Qu.: 0.9970	3rd Qu.: 0.4000	3rd Qu.: 0.3900
Max. : 14.90	Max. : 1.0390	Max. : 1.5800	Max. : 1.6600

잔류_당	품질도
Min. : 0.600	기본품질: 2384
1st Qu.: 1.800	우수품질: 4113
Median : 3.000	
Mean : 5.443	
3rd Qu.: 8.100	
Max. : 65.800	

본격적인 데이터 전처리를 진행하기 전 boxplot을 이용해 변수의 이상값을 확인해보았습니다. 그 결과 잔류당의 이상값 범위가 넓은 것을 확인할 수 있었고 잔류당과 구연산의 이상값을 제거하기로 결정했습니다.



[데이터 전처리 전 이상값 확인 모습]

다음은 본격적인 데이터 전처리 진행 과정입니다. **알코올**은 단위가 다른 변수들과 달라 **표준화**를 진행하였고 **밀도**는 **최대-최소 스케일링**, **휘발성 산도**는 **로그변환**, **구연산**은 **이상값 제거 후 표준화**, 마지막으로 **잔류당**은 **이상값 제거 후 로그 변환 및 최대-최소 스케일링**을 진행하였습니다. 이렇게 전처리를 진행한 후 데이터를 살펴보았는데 음수 범위 값들이 존재해 데이터를 해석하는 데 어려움이 있을 것이라 판단해 모든 변수들을 양수화 하였습니다.

```
#1. 알코올: 표준화 후 양수로 변환
wine_quality <- wine_quality %>%
  mutate(알코올 = scale(알코올)) %>%
  mutate(알코올 = 알코올 - min(알코올) + 1)

# 2. 밀도: 최소-최대 스케일링 (양수 범위)
wine_quality <- wine_quality %>%
  mutate(밀도 = (밀도 - min(밀도)) / (max(밀도) - min(밀도)))

# 3. 휘발성 산도: 로그 변환 후 양수로 보정
wine_quality <- wine_quality %>%
  mutate(휘발성_산도 = log(휘발성_산도 + 1)) %>%
  mutate(휘발성_산도 = 휘발성_산도 - min(휘발성_산도) + 1)

# 4. 구연산: 이상값 제거 후 표준화 및 양수 변환
remove_outliers <- function(data, column) {
  Q1 <- quantile(data[[column]], 0.25)
  Q3 <- quantile(data[[column]], 0.75)
  IQR <- Q3 - Q1
  lower_bound <- Q1 - 1.5 * IQR
  upper_bound <- Q3 + 1.5 * IQR
  data <- data %>% filter(data[[column]] >= lower_bound & data[[column]] <= upper_bound)
  return(data)
}

wine_quality <- remove_outliers(wine_quality, "구연산")
wine_quality <- wine_quality %>%
  mutate(구연산 = scale(구연산)) %>%
  mutate(구연산 = 구연산 - min(구연산) + 1)

# 5. 잔류 당: 이상값 제거 후 로그 변환 및 최소-최대 스케일링 (양수 범위)
wine_quality <- remove_outliers(wine_quality, "잔류_당")
wine_quality <- wine_quality %>%
  mutate(잔류_당 = log(잔류_당 + 1)) %>%
  mutate(잔류_당 = (잔류_당 - min(잔류_당)) / (max(잔류_당) - min(잔류_당))) %>%
  mutate(잔류_당 = 잔류_당 + 1) # 양수 범위로 조정

# 결과 확인
summary(wine_quality)
```


데이터 전처리 후 데이터들을 살펴보면 아래와 같은 결과를 얻을 수 있습니다.

알코올.V1	밀도	휘발성_산도	구연산.V1
Min. :1.000000	Min. :0.00000	Min. :1.000	Min. :1.000000
1st Qu.:2.257638	1st Qu.:0.09635	1st Qu.:1.122	1st Qu.:2.990873
Median :3.012221	Median :0.14440	Median :1.170	Median :3.443344
Mean :3.116621	Mean :0.14208	Mean :1.199	Mean :3.514704
3rd Qu.:3.766804	3rd Qu.:0.18590	3rd Qu.:1.245	3rd Qu.:4.076804
Max. :6.198238	Max. :0.31965	Max. :1.730	Max. :6.067677

잔류_당	품질도
Min. :1.000	기본품질:2075
1st Qu.:1.227	우수품질:3813
Median :1.382	
Mean :1.463	
3rd Qu.:1.702	
Max. :2.000	

데이터들을 시각화한 코드는 아래와 같습니다.

```
#데이터 전처리 후 각 변수의 시각화 (히스토그램)

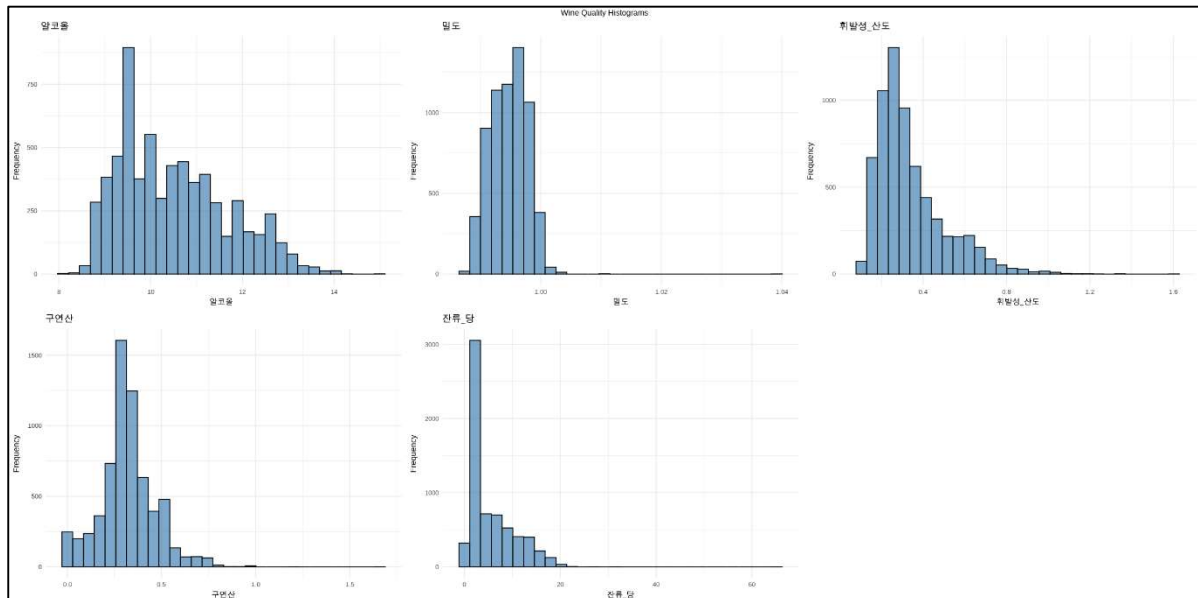
# 숫자형 변수 선택
numeric_vars <- names(wine_quality)[sapply(wine_quality, is.numeric)]

# 히스토그램 저장할 리스트 생성
plots <- list()

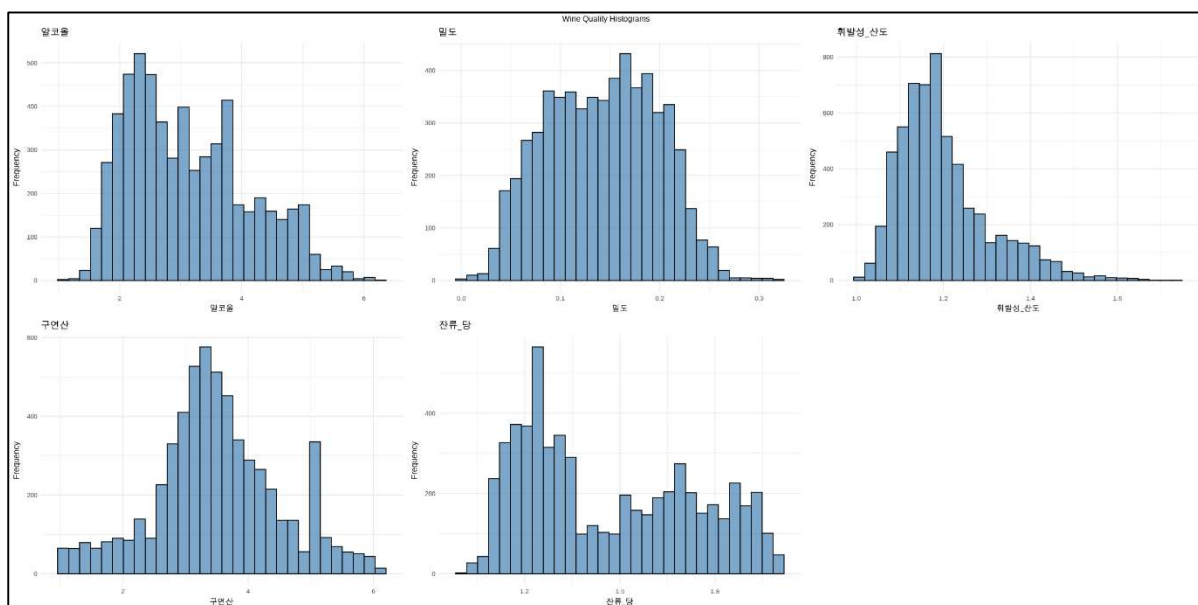
for (var in numeric_vars) {
  p <- ggplot(wine_quality, aes(x = .data[[var]])) + # aes()와 tidy eval 사용
    geom_histogram(bins = 30, fill = "steelblue", color = "black", alpha = 0.7) +
    labs(title = var, x = var, y = "Frequency") +
    theme_minimal() +
    theme(
      plot.title = element_text(size = 14), # 제목 크기 조정
      axis.title = element_text(size = 12), # 축 제목 크기 조정
      axis.text = element_text(size = 10) # 축 값 크기 조정
    )
  plots[[var]] <- p
}

# 그래프 크기 조정 및 레이아웃 변경
do.call(grid.arrange, c(plots, ncol = 3, top = "Wine Quality Histograms"))
```

데이터 전처리 전 후의 분포를 비교하기 위해 시각화 한 그래프를 아래에 같이 첨부했습니다. 상단이 데이터 전처리 전의 시각화 모습이고 하단의 그래프가 데이터 전처리 후의 모습입니다.



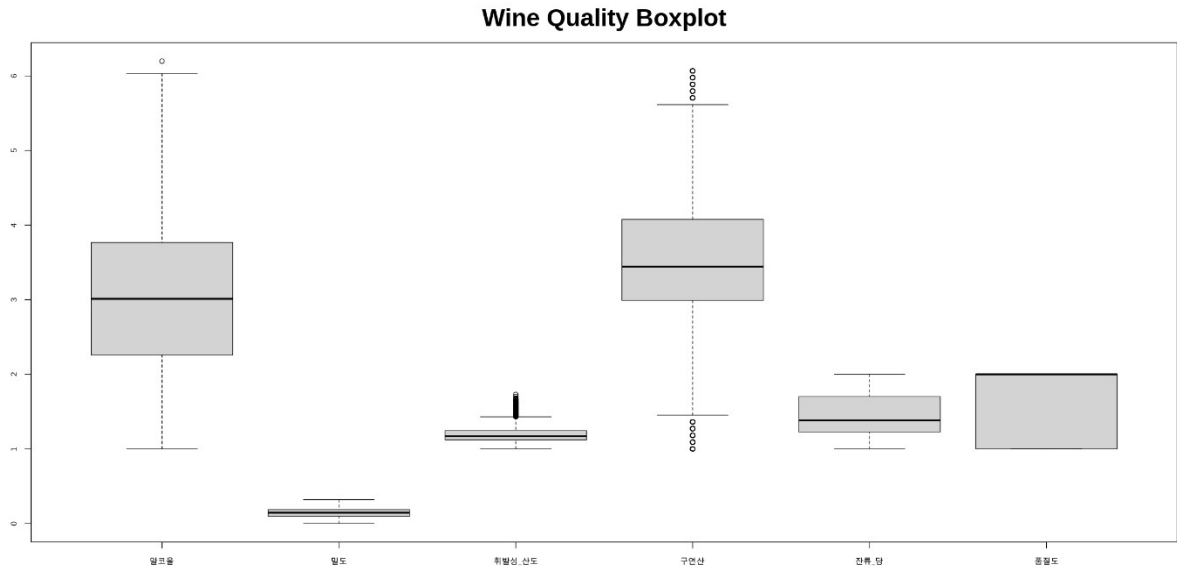
<데이터 전처리 전 시각화>



<데이터 전처리 후 시각화>

데이터 전처리 전·후의 모습을 비교했을 때 전처리 후의 변수들의 모습이 더 균일한 것을 확인할 수 있습니다. 또한, 변수 간의 범위가 조정되어 모델 학습을 진행할 때 특정 변수가 지나치게 영향을 미치는 문제를 발생시키지 않을 것이라 판단됩니다.

데이터 전처리 후 boxplot의 모습을 살펴보면 아래와 같습니다. 각 변수에서 이상값의 영향을 최소화하였으며 데이터가 균일하고 안정된 분포를 가지게 되었음을 확인할 수 있었습니다.



[데이터 전처리 후 이상값 확인 모습]

5. 가설 수립

저희는 데이터 상관관계 분석과 중요도 분석 후 얻은 알코올, 밀도, 휘발성 산도, 구연산 잔류당의 5개 변수를 설명변수로 설정하고 반응 변수를 품질도 변수로 설정하였습니다. 이를 바탕으로 "와인의 품질도는 설명 변수와 관련이 없다."를 영가설로 설정하고 그와 대립되는 "와인의 품질도는 설명 변수와 관련이 있다."를 대립 가설로 설정했습니다.

6. 모델링

우선 모델링을 진행하기 전에 train 데이터와 test 데이터를 분리하였습니다. train 데이터를 75비율로 test 데이터는 25의 비율로 설정하였습니다.

```
# 데이터 분리 (75% 훈련, 25% 테스트)
set.seed(123) # 재현성을 위해 설정
train_index <- createDataPartition(wine_quality$품질도, p = 0.75, list = FALSE)
train_data <- wine_quality[train_index, ]
test_data <- wine_quality[-train_index, ]

# 설명변수와 반응변수 분리
x_train <- train_data[, c("알코올", "밀도", "휘발성_산도", "구연산", "잔류_당")]
y_train <- train_data$품질도
x_test <- test_data[, c("알코올", "밀도", "휘발성_산도", "구연산", "잔류_당")]
y_test <- test_data$품질도
```

1) 로지스틱 회귀

```
# 로지스틱 회귀
# 모델 학습
log_model <- glm(품질도 ~ 알코올 + 밀도 + 휘발성_산도 + 구연산 + 잔류_당,
  data = train_data, family = binomial)

# 예측
log_pred_prob <- predict(log_model, newdata = x_test, type = "response")
log_pred <- ifelse(log_pred_prob > 0.5, levels(y_train)[2], levels(y_train)[1])

# 정확도 및 F1-score 계산
log_accuracy <- mean(log_pred == y_test)
log_f1 <- F1_Score(y_pred = log_pred, y_true = y_test, positive = levels(y_test)[2])

# Confusion Matrix 생성
log_conf_matrix <- confusionMatrix(as.factor(log_pred), as.factor(y_test))

# 출력
cat("로지스틱 회귀 Accuracy:", log_accuracy, "\n")
cat("로지스틱 회귀 F1-Score:", log_f1, "\n")
print(log_conf_matrix)
```

로지스틱 회귀 Accuracy: 0.757308
로지스틱 회귀 F1-Score: 0.8257687
Confusion Matrix and Statistics

	Reference	
Prediction	기본품질	우수품질
기본품질	268	107
우수품질	250	846

Accuracy : 0.7573
95% CI : (0.7345, 0.779)
No Information Rate : 0.6479
P-Value [Acc > NIR] : < 2.2e-16

Kappa : 0.4323

Mcnemar's Test P-Value : 5.672e-14

Sensitivity : 0.5174
Specificity : 0.8877
Pos Pred Value : 0.7147
Neg Pred Value : 0.7719
Prevalence : 0.3521
Detection Rate : 0.1822
Detection Prevalence : 0.2549
Balanced Accuracy : 0.7025

'Positive' Class : 기본품질

2) Decision Tree

```
# Decision Tree
library(rpart)
library(rpart.plot)
library(MLmetrics)
library(caret)

# 모델 학습
tree_model <- rpart(품질도 ~ 알코올 + 밀도 + 휘발성_산도 + 구연산 + 잔류_당,
  data = train_data, method = "class")

# 예측
tree_pred <- predict(tree_model, newdata = x_test, type = "class")

# 정확도 및 F1-score 계산
tree_accuracy <- mean(tree_pred == y_test)
tree_f1 <- F1_Score(y_pred = tree_pred, y_true = y_test, positive = levels(y_test)[2])

cat("Decision Tree Accuracy:", tree_accuracy, "\n")
cat("Decision Tree F1-Score:", tree_f1, "\n")

# Confusion Matrix 계산
confusion_matrix <- confusionMatrix(factor(tree_pred, levels = levels(y_test)), y_test)
print(confusion_matrix)

# 의사결정나무 시각화
rpart.plot(tree_model)
```

Decision Tree Accuracy: 0.7539089
Decision Tree F1-Score: 0.8200795
Confusion Matrix and Statistics

	Reference	
Prediction	기본품질	우수품질
기본품질	284	128
우수품질	234	825

Accuracy : 0.7539
95% CI : (0.7311, 0.7757)
No Information Rate : 0.6479
P-Value [Acc > NIR] : < 2.2e-16

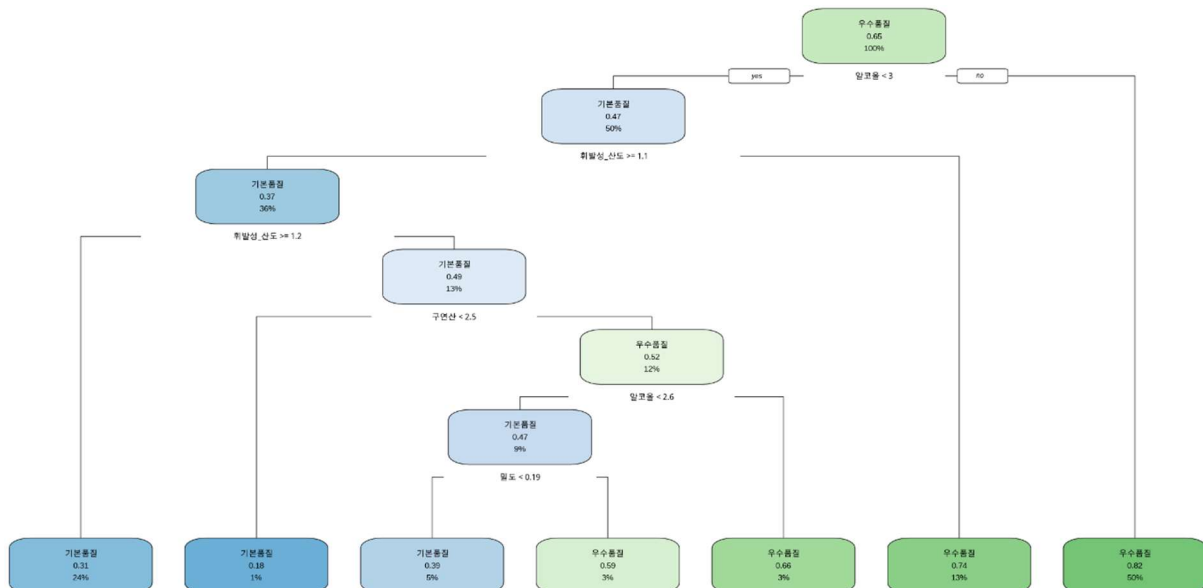
Kappa : 0.4342

Mcnemar's Test P-Value : 3.416e-08

Sensitivity : 0.5483
Specificity : 0.8657
Pos Pred Value : 0.6893
Neg Pred Value : 0.7790
Prevalence : 0.3521
Detection Rate : 0.1931
Detection Prevalence : 0.2801
Balanced Accuracy : 0.7070

'Positive' Class : 기본품질

다음으로는 **Decision Tree**를 시각화한 결과입니다. 시각화한 모습을 보면 모델이 데이터에서 발견한 규칙을 알 수 있고, 어떤 변수가 품질 예측에 큰 영향을 미치는지 알 수 있습니다. 예를 들어, 휘발성 산도가 1.2 이하면서 구연산이 특정 값 이하일 경우 기본 품질로 분류되는 경향을 확인할 수 있었습니다. 알코올은 값이 클수록 우수 품질로 분류되는 경향이 보였습니다.



3) Bagging

```
# Bagging 모델링
library(ipred)
library(caret)
library(MLmetrics)
```

```
# Bagging 모델 학습
bagging_model <- bagging(
  품질도 ~ 알코올 + 밀도 + 휘발성_산도 + 구연산 + 잔류_당,
  data = train_data,
  nbagg = 50 # Bagging 반복 횟수
)

# 예측 및 평가
bagging_preds <- predict(bagging_model, test_data, type = "class")
bagging_accuracy <- mean(bagging_preds == test_data$품질도)
bagging_f1 <- F1_Score(y_true = test_data$품질도, y_pred = bagging_preds, positive = levels(test_data$품질도)[2])

# Confusion Matrix 생성
conf_matrix <- confusionMatrix(
  as.factor(bagging_preds),
  as.factor(test_data$품질도),
  positive = levels(test_data$품질도)[2]
)

# 결과 출력
cat("Bagging Accuracy:", bagging_accuracy, "\n")
cat("Bagging F1 Score:", bagging_f1, "\n")
print(conf_matrix)
```

```
Bagging Accuracy: 0.800136
Bagging F1 Score: 0.8481405
Confusion Matrix and Statistics

      Reference
Prediction 기본품질 우수품질
기본품질    356    132
우수품질    162    821

      Accuracy : 0.8001
      95% CI : (0.7788, 0.8203)
      No Information Rate : 0.6479
      P-Value [Acc > NIR] : < 2e-16

      Kappa : 0.5561

      McNemar's Test P-Value : 0.09078

      Sensitivity : 0.8615
      Specificity : 0.6873
      Pos Pred Value : 0.8352
      Neg Pred Value : 0.7295
      Prevalence : 0.6479
      Detection Rate : 0.5581
      Detection Prevalence : 0.6683
      Balanced Accuracy : 0.7744

      'Positive' Class : 우수품질
```

4) Random Forest

```
# Random Forest
library(randomForest)
library(caret)
library(MLmetrics)

# 모델 학습
rf_model <- randomForest(품질도 ~ 알코올 + 밀도 + 휘발성_산도 + 구연산 + 잔류_당,
                        data = train_data)

# 예측
rf_pred <- predict(rf_model, newdata = x_test)

# 정확도 및 F1-score 계산
rf_accuracy <- mean(rf_pred == y_test)
rf_f1 <- F1_Score(y_pred = rf_pred, y_true = y_test, positive = levels(y_test)[2])

cat("Random Forest Accuracy:", rf_accuracy, "\n")
cat("Random Forest F1-Score:", rf_f1, "\n")

# Confusion Matrix 생성
rf_conf_matrix <- confusionMatrix(as.factor(rf_pred), as.factor(y_test))

# Confusion Matrix 출력
cat("\nRandom Forest Confusion Matrix:\n")
print(rf_conf_matrix)
```

Random Forest Accuracy: 0.8082937
Random Forest F1-Score: 0.8553846

Random Forest Confusion Matrix:
Confusion Matrix and Statistics

	Reference	
Prediction	기본품질	우수품질
기본품질	355	119
우수품질	163	834

Accuracy : 0.8083
95% CI : (0.7872, 0.8281)
No Information Rate : 0.6479
P-Value [Acc > NIR] : < 2e-16

Kappa : 0.5715

Mcnemar's Test P-Value : 0.01045

Sensitivity : 0.6853
Specificity : 0.8751
Pos Pred Value : 0.7489
Neg Pred Value : 0.8365
Prevalence : 0.3521
Detection Rate : 0.2413
Detection Prevalence : 0.3222
Balanced Accuracy : 0.7802

'Positive' Class : 기본품질

5) SVM

```
# SVM
library(e1071)
library(caret)
library(MLmetrics)

# 모델 학습
svm_model <- svm(품질도 ~ 알코올 + 밀도 + 휘발성_산도 + 구연산 + 잔류_당,
                data = train_data, kernel = "radial")

# 예측
svm_pred <- predict(svm_model, newdata = x_test)

# 정확도 및 F1-score 계산
svm_accuracy <- mean(svm_pred == y_test)
svm_f1 <- F1_Score(y_pred = svm_pred, y_true = y_test, positive = levels(y_test)[2])

cat("SVM Accuracy:", svm_accuracy, "\n")
cat("SVM F1-Score:", svm_f1, "\n")

# Confusion Matrix 생성
svm_conf_matrix <- confusionMatrix(as.factor(svm_pred), as.factor(y_test))

# Confusion Matrix 출력
cat("\nSVM Confusion Matrix:\n")
print(svm_conf_matrix)
```

SVM Accuracy: 0.7539089
SVM F1-Score: 0.8222004

SVM Confusion Matrix:
Confusion Matrix and Statistics

	Reference	
Prediction	기본품질	우수품질
기본품질	272	116
우수품질	246	837

Accuracy : 0.7539
95% CI : (0.7311, 0.7757)
No Information Rate : 0.6479
P-Value [Acc > NIR] : < 2.2e-16

Kappa : 0.4279

Mcnemar's Test P-Value : 1.201e-11

Sensitivity : 0.5251
Specificity : 0.8783
Pos Pred Value : 0.7010
Neg Pred Value : 0.7729
Prevalence : 0.3521
Detection Rate : 0.1849
Detection Prevalence : 0.2638
Balanced Accuracy : 0.7017

'Positive' Class : 기본품질

6) K-NN

```
# k-NN

# 데이터 스케일링
x_train_scaled <- scale(x_train)
x_test_scaled <- scale(x_test)

# k-NN 모델 학습 및 예측
k <- 5 # k 값 설정
knn_pred <- knn(train = x_train_scaled, test = x_test_scaled, cl = y_train, k = k)

# 정확도 및 F1-score 계산
knn_accuracy <- mean(knn_pred == y_test)
knn_f1 <- F1_Score(y_pred = knn_pred, y_true = y_test, positive = levels(y_test)[2])

cat("k-NN Accuracy:", knn_accuracy, "\n")
cat("k-NN F1-Score:", knn_f1, "\n")

# Confusion Matrix 생성
knn_conf_matrix <- confusionMatrix(as.factor(knn_pred), as.factor(y_test))

# Confusion Matrix 출력
cat("\nk-NN Confusion Matrix:\n")
print(knn_conf_matrix)
```

k-NN Accuracy: 0.7362339
k-NN F1-Score: 0.7991718

k-NN Confusion Matrix:
Confusion Matrix and Statistics

	Reference	
Prediction	기본품질	우수품질
기본품질	311	181
우수품질	207	772

Accuracy : 0.7362
95% CI : (0.7129, 0.7586)
No Information Rate : 0.6479
P-Value [Acc > NIR] : 2.412e-13

Kappa : 0.4152

Mcnemar's Test P-Value : 0.2044

Sensitivity : 0.6004
Specificity : 0.8101
Pos Pred Value : 0.6321
Neg Pred Value : 0.7886
Prevalence : 0.3521
Detection Rate : 0.2114
Detection Prevalence : 0.3345
Balanced Accuracy : 0.7052

'Positive' Class : 기본품질

6 개의 모델로 진행한 모델링의 결과를 비교하면 아래와 같은 결과를 얻을 수 있습니다.

```
# 결과 비교 테이블
results <- data.frame(
  Model = c("Logistic Regression", "Decision Tree", "Bagging", "Random Forest", "SVM", "k-NN"),
  Accuracy = c(log_accuracy, tree_accuracy, bagging_accuracy, rf_accuracy, svm_accuracy, knn_accuracy),
  F1_Score = c(log_f1, tree_f1, bagging_f1, rf_f1, svm_f1, knn_f1)
)

print(results)
```

	Model	Accuracy	F1_Score
1	Logistic Regression	0.7573080	0.8257687
2	Decision Tree	0.7539089	0.8200795
3	Bagging	0.8001360	0.8481405
4	Random Forest	0.8082937	0.8553846
5	SVM	0.7539089	0.8222004
6	k-NN	0.7362339	0.7991718

Random Forest, Bagging, 로지스틱 회귀, SVM, Decision Tree, K-NN 순으로 모델의 성능이 좋은 것을 확인할 수 있습니다.

여기까지 프로젝트 결과 발표 때 진행한 내용입니다. 질의응답 시간에 “모델링을 수행할 때 튜닝 과정부분”을 생각하지 못하여 발표 후 튜닝 과정을 진행하여 모델링을 다시 수행한 결과를 아래와 같이 얻을 수 있었습니다.

1) 로지스틱 회귀

```
# 로지스틱 회귀

# 튜닝 및 모델 학습
log_tune <- train(
  품질도 ~ 알코올 + 밀도 + 휘발성_산도 + 구연산 + 잔류_당,
  data = train_data,
  method = "glm",
  family = "binomial",
  trControl = trainControl(method = "cv", number = 5) # 5-fold 교차검증
)

# 최적 모델 출력
print(log_tune)

# 예측
log_pred <- predict(log_tune, newdata = x_test)

# 정확도 및 F1-score 계산
log_accuracy <- mean(log_pred == y_test)
log_f1 <- F1_Score(y_pred = log_pred, y_true = y_test, positive = levels(y_test)[2])

# Confusion Matrix 생성
log_conf_matrix <- confusionMatrix(as.factor(log_pred), as.factor(y_test))

# 출력
cat("로지스틱 회귀 튜닝 후 Accuracy:", log_accuracy, "\n")
cat("로지스틱 회귀 튜닝 후 F1-Score:", log_f1, "\n")
print(log_conf_matrix)
```

Generalized Linear Model

4417 samples
5 predictor
2 classes: '기본품질', '우수품질'

No pre-processing
Resampling: Cross-Validated (5 fold)
Summary of sample sizes: 3534, 3533, 3534, 3534, 3533
Resampling results:

Accuracy	Kappa
0.7457552	0.4124534

로지스틱 회귀 튜닝 후 Accuracy: 0.757308
로지스틱 회귀 튜닝 후 F1-Score: 0.8257687
Confusion Matrix and Statistics

	Reference	
Prediction	기본품질	우수품질
기본품질	268	107
우수품질	250	846

Accuracy : 0.7573

95% CI : (0.7346, 0.779)

No Information Rate : 0.6479

P-Value [Acc > NIR] : < 2.2e-16

Kappa : 0.4323

McNemar's Test P-Value : 5.672e-14

Sensitivity : 0.5174

Specificity : 0.8877

Pos Pred Value : 0.7147

Neg Pred Value : 0.7719

Prevalence : 0.3521

Detection Rate : 0.1822

Detection Prevalence : 0.2549

Balanced Accuracy : 0.7025

'Positive' Class : 기본품질

2) Decision Tree

```
# 필요한 라이브러리 로드
library(rpart)

# 튜닝 및 모델 학습
tree_tune <- train(
  품질도 ~ 알코올 + 밀도 + 휘발성_산도 + 구연산 + 잔류_당,
  data = train_data,
  method = "rpart",
  trControl = trainControl(method = "cv", number = 5), # 5-fold 교차검증
  tuneGrid = expand.grid(cp = seq(0.01, 0.1, by = 0.01)) # cp 파라미터 튜닝
)

# 최적 모델 확인
print(tree_tune)

# 예측
tree_pred <- predict(tree_tune, newdata = x_test)
```

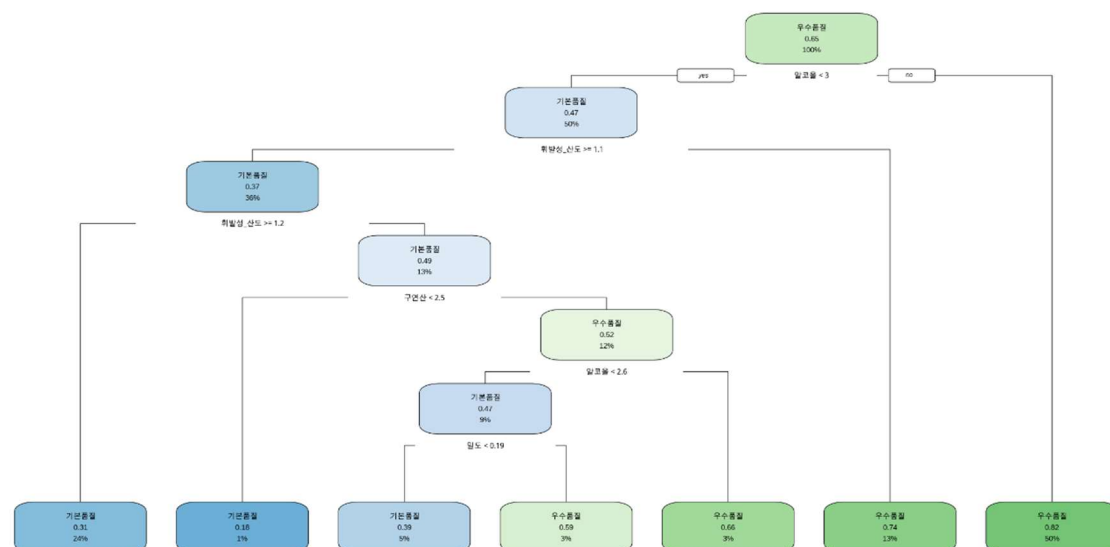
```
# 정확도 및 F1-score 계산
tree_accuracy <- mean(tree_pred == y_test)
tree_f1 <- F1_Score(y_pred = tree_pred, y_true = y_test, positive = levels(y_test)[2])

# Confusion Matrix 계산
confusion_matrix <- confusionMatrix(factor(tree_pred, levels = levels(y_test)), y_test)

# 출력
cat("Decision Tree 튜닝 후 Accuracy:", tree_accuracy, "\n")
cat("Decision Tree 튜닝 후 F1-Score:", tree_f1, "\n")
print(confusion_matrix)

# 최적 모델 시각화
rpart.plot(tree_tune$finalModel)
```

CART	Reference
4417 samples 5 predictor 2 classes: '기본품질', '우수품질'	Prediction 기본품질 우수품질 기본품질 284 128 우수품질 234 825
No pre-processing Resampling: Cross-Validated (5 fold) Summary of sample sizes: 3534, 3534, 3533, 3534, 3533 Resampling results across tuning parameters:	Accuracy : 0.7539 95% CI : (0.7311, 0.7757) No Information Rate : 0.6479 P-Value [Acc > NIR] : < 2.2e-16 Kappa : 0.4342 McNemar's Test P-Value : 3.416e-08 Sensitivity : 0.5483 Specificity : 0.8657 Pos Pred Value : 0.6893 Neg Pred Value : 0.7790 Prevalence : 0.3521 Detection Rate : 0.1931 Detection Prevalence : 0.2801 Balanced Accuracy : 0.7070 'Positive' Class : 기본품질
cp Accuracy Kappa 0.01 0.7373746 0.4007211 0.02 0.7362421 0.4078410 0.03 0.7362421 0.4078410 0.04 0.7362421 0.4078410 0.05 0.7362421 0.4078410 0.06 0.7362421 0.4078410 0.07 0.7362421 0.4078410 0.08 0.7362421 0.4078410 0.09 0.7362421 0.4078410 0.10 0.7362421 0.4078410	
Accuracy was used to select the optimal model using the largest value. The final value used for the model was cp = 0.01. Decision Tree 튜닝 후 Accuracy: 0.7539089 Decision Tree 튜닝 후 F1-Score: 0.8200795 Confusion Matrix and Statistics	



3) Bagging

```
# 필요한 라이브러리 로드
library(ipred)
library(caret)
library(MLmetrics)

# Bagging 모델 학습 (튜닝 포함)
set.seed(123) # 재현 가능성 설정
bagging_tune <- train(
  품질도 ~ 알코올 + 밀도 + 휘발성_산도 + 구연산 + 잔류_당,
  data = train_data,
  method = "treebag", # Bagging 기반 트리 모델
  trControl = trainControl(method = "cv", number = 5) # 5-fold 교차검증
)

# 최적 모델 확인
print(bagging_tune)

# 예측 및 평가
bagging_preds <- predict(bagging_tune, newdata = test_data)
bagging_accuracy <- mean(bagging_preds == test_data$품질도)
bagging_f1 <- F1_Score(y_true = test_data$품질도, y_pred = bagging_preds, positive = levels(test_data$품질도)[2])

# Confusion Matrix 생성
conf_matrix <- confusionMatrix(
  as.factor(bagging_preds),
  as.factor(test_data$품질도),
  positive = levels(test_data$품질도)[2]
)

# 결과 출력
cat("Bagging 튜닝 후 Accuracy:", bagging_accuracy, "\n")
cat("Bagging 튜닝 후 F1 Score:", bagging_f1, "\n")
print(conf_matrix)
```

Bagged CART

4417 samples
5 predictor
2 classes: '기본품질', '우수품질'

No pre-processing
Resampling: Cross-Validated (5 fold)
Summary of sample sizes: 3534, 3534, 3533, 3534, 3533
Resampling results:

Accuracy	Kappa
0.7860525	0.5204527

Bagging 튜닝 후 Accuracy: 0.8021754
Bagging 튜닝 후 F1 Score: 0.8505393
Confusion Matrix and Statistics

	Reference	
Prediction	기본품질	우수품질
기본품질	352	125
우수품질	166	828

Accuracy : 0.8022
95% CI : (0.7809, 0.8223)
No Information Rate : 0.6479
P-Value [Acc > NIR] : < 2e-16

Kappa : 0.5585

Mcnemar's Test P-Value : 0.01904

Sensitivity : 0.8688
Specificity : 0.6795
Pos Pred Value : 0.8330
Neg Pred Value : 0.7379
Prevalence : 0.6479
Detection Rate : 0.5629
Detection Prevalence : 0.6757
Balanced Accuracy : 0.7742

'Positive' Class : 우수품질

4) Random Forest

```
# 필요한 라이브러리 로드
library(randomForest)
library(caret)
library(MLmetrics)

# Random Forest 모델 학습 (튜닝 포함)
set.seed(123) # 재현 가능성을 위해 설정
rf_tune <- train(
  품질도 ~ 알코올 + 밀도 + 휘발성_산도 + 구연산 + 잔류_당,
  data = train_data,
  method = "rf", # Random Forest 모델
  tuneGrid = expand.grid(mtry = c(2, 3, 4)), # mtry 값 튜닝
  trControl = trainControl(method = "cv", number = 5) # 5-fold 교차검증
)

# 최적 모델 확인
print(rf_tune)

# 예측
rf_pred <- predict(rf_tune, newdata = x_test)

# 정확도 및 F1-score 계산
rf_accuracy <- mean(rf_pred == y_test)
rf_f1 <- F1_Score(y_pred = rf_pred, y_true = y_test, positive = levels(y_test)[2])

# Confusion Matrix 생성
rf_conf_matrix <- confusionMatrix(as.factor(rf_pred), as.factor(y_test))

# 결과 출력
cat("Random Forest 튜닝 후 Accuracy:", rf_accuracy, "\n")
cat("Random Forest 튜닝 후 F1-Score:", rf_f1, "\n")
cat("\nRandom Forest Confusion Matrix:\n")
print(rf_conf_matrix)
```

Random Forest

4417 samples
5 predictor
2 classes: '기본품질', '우수품질'

No pre-processing
Resampling: Cross-Validated (5 fold)
Summary of sample sizes: 3534, 3534, 3533, 3534, 3533
Resampling results across tuning parameters:

mtry	Accuracy	Kappa
2	0.7996398	0.5486021
3	0.7964690	0.5413856
4	0.7951120	0.5387974

Accuracy was used to select the optimal model using the largest value.
The final value used for the model was mtry = 2.
Random Forest 튜닝 후 Accuracy: 0.8069341
Random Forest 튜닝 후 F1-Score: 0.8546571

Random Forest Confusion Matrix:
Confusion Matrix and Statistics

	Reference		
Prediction	기본품질	우수품질	
기본품질	352	118	
우수품질	166	835	

Accuracy : 0.8069
95% CI : (0.7858, 0.8268)
No Information Rate : 0.6479
P-Value [Acc > NIR] : < 2.2e-16

Kappa : 0.5677

Mcnemar's Test P-Value : 0.005288

Sensitivity : 0.6795
Specificity : 0.8762
Pos Pred Value : 0.7489
Neg Pred Value : 0.8342
Prevalence : 0.3521
Detection Rate : 0.2393
Detection Prevalence : 0.3195
Balanced Accuracy : 0.7779

'Positive' Class : 기본품질

5) SVM

```
# 필요한 라이브러리 로드
library(e1071)
library(caret)
library(MLmetrics)

# SVM 모델 학습 (튜닝 포함)
set.seed(123) # 재현 가능성을 위해 설정
svm_tune <- train(
  품질도 ~ 알코올 + 밀도 + 휘발성_산도 + 구연산 + 잔류_당,
  data = train_data,
  method = "svmRadial", # Radial Kernel SVM
  tuneGrid = expand.grid(
    C = c(0.1, 1, 10), # Regularization parameter
    sigma = c(0.01, 0.1, 1) # RBF kernel parameter
  ),
  trControl = trainControl(method = "cv", number = 5) # 5-fold 교차검증
)

# 최적 모델 확인
print(svm_tune)

# 예측
svm_pred <- predict(svm_tune, newdata = x_test)

# 정확도 및 F1-score 계산
svm_accuracy <- mean(svm_pred == y_test)
svm_f1 <- F1_Score(y_pred = svm_pred, y_true = y_test, positive = levels(y_test)[2])

# Confusion Matrix 생성
svm_conf_matrix <- confusionMatrix(as.factor(svm_pred), as.factor(y_test))

# 결과 출력
cat("SVM 튜닝 후 Accuracy:", svm_accuracy, "\n")
cat("SVM 튜닝 후 F1-Score:", svm_f1, "\n")
cat("\nSVM Confusion Matrix:\n")
print(svm_conf_matrix)
```

Support Vector Machines with Radial Basis Function Kernel

4417 samples
5 predictor
2 classes: '기본품질', '우수품질'

No pre-processing
Resampling: Cross-Validated (5 fold)
Summary of sample sizes: 3534, 3534, 3533, 3534, 3533
Resampling results across tuning parameters:

C	sigma	Accuracy	Kappa
0.1	0.01	0.6995701	0.2437545
0.1	0.10	0.7369273	0.3846701
0.1	1.00	0.7400975	0.3851107
1.0	0.01	0.7391897	0.3816850
1.0	0.10	0.7473435	0.4163754
1.0	1.00	0.7539097	0.4359170
10.0	0.01	0.7432665	0.3991334
10.0	0.10	0.7514187	0.4270250
10.0	1.00	0.7480220	0.4278055

Accuracy was used to select the optimal model using the largest value.
The final values used for the model were sigma = 1 and C = 1.
SVM 튜닝 후 Accuracy: 0.7777022
SVM 튜닝 후 F1-Score: 0.8373943

SVM Confusion Matrix:
Confusion Matrix and Statistics
Reference

Prediction	기본품질	우수품질
기본품질	302	111
우수품질	216	842

Accuracy : 0.7777
95% CI : (0.7556, 0.7987)
No Information Rate : 0.6479
P-Value [Acc > NIR] : < 2.2e-16

Kappa : 0.4892

Mcnemar's Test P-Value : 8.861e-09

Sensitivity : 0.5830
Specificity : 0.8835
Pos Pred Value : 0.7312
Neg Pred Value : 0.7958
Prevalence : 0.3521
Detection Rate : 0.2053
Detection Prevalence : 0.2808
Balanced Accuracy : 0.7333

'Positive' Class : 기본품질

6) K-NN

```
# 필요한 라이브러리 로드
library(class)
library(caret)
library(MLmetrics)

# 데이터 스케일링
x_train_scaled <- scale(x_train)
x_test_scaled <- scale(x_test)

# k-NN 모델 튜닝
set.seed(123) # 재현 가능성을 위해 설정
k_values <- seq(1, 15, by = 2) # k 값 후보 설정
knn_results <- data.frame(k = k_values, Accuracy = NA, F1_Score = NA)

for (k in k_values) {
  # 모델 학습 및 예측
  knn_pred <- knn(train = x_train_scaled, test = x_test_scaled, cl = y_train, k = k)

  # 정확도 및 F1-score 계산
  accuracy <- mean(knn_pred == y_test)
  f1 <- F1_Score(y_pred = knn_pred, y_true = y_test, positive = levels(y_test)[2])

  # 결과 저장
  knn_results[knn_results$k == k, ] <- c(k, accuracy, f1)
}

# 최적 k 값 선택
best_k <- knn_results[which.max(knn_results$F1_Score), "k"]

# 최적 k 값으로 모델 재학습 및 예측
knn_pred <- knn(train = x_train_scaled, test = x_test_scaled, cl = y_train, k = best_k)

# 정확도 및 F1-score 계산
knn_accuracy <- mean(knn_pred == y_test)
knn_f1 <- F1_Score(y_pred = knn_pred, y_true = y_test, positive = levels(y_test)[2])

# Confusion Matrix 생성
knn_conf_matrix <- confusionMatrix(as.factor(knn_pred), as.factor(y_test))

# 결과 출력
cat("k-NN (최적 k =", best_k, ") Accuracy:", knn_accuracy, "\n")
cat("k-NN (최적 k =", best_k, ") F1-Score:", knn_f1, "\n")
cat("\nk-NN Confusion Matrix:\n")
print(knn_conf_matrix)
```

k-NN (최적 k = 1) Accuracy: 0.7681849
k-NN (최적 k = 1) F1-Score: 0.8206207

k-NN Confusion Matrix:
Confusion Matrix and Statistics

	Reference	
Prediction	기본품질	우수품질
기본품질	350	173
우수품질	168	780

Accuracy : 0.7682
95% CI : (0.7458, 0.7895)
No Information Rate : 0.6479
P-Value [Acc > NIR] : <2e-16

Kappa : 0.4931

Mcnemar's Test P-Value : 0.8285

Sensitivity : 0.6757
Specificity : 0.8185
Pos Pred Value : 0.6692
Neg Pred Value : 0.8228
Prevalence : 0.3521
Detection Rate : 0.2379
Detection Prevalence : 0.3555
Balanced Accuracy : 0.7471

'Positive' Class : 기본품질

6 개의 모델을 튜닝까지 거친 후 실행한 결과를 비교한 테이블은 아래와 같습니다.
왼쪽 사진은 튜닝 전의 결과이고 오른쪽은 튜닝 후의 결과입니다.

로지스틱 회귀와 Decision Tree 는 모델의 성능이 같았지만 그 외의 모델들은 튜닝 후의 성능이 더 나아진 것을 확인할 수 있었습니다.

	Model	Accuracy	F1_Score
1	Logistic Regression	0.7573080	0.8257687
2	Decision Tree	0.7539089	0.8200795
3	Bagging	0.8001360	0.8481405
4	Random Forest	0.8082937	0.8553846
5	SVM	0.7539089	0.8222004
6	k-NN	0.7362339	0.7991718

튜닝 전 결과

	Model	Accuracy	F1_Score
1	Logistic Regression	0.7573080	0.8257687
2	Decision Tree	0.7539089	0.8200795
3	Bagging	0.8021754	0.8505393
4	Random Forest	0.8069341	0.8546571
5	SVM	0.7777022	0.8373943
6	k-NN	0.7681849	0.8206207

튜닝 후 결과

7. 결론

저희는 이 모델을 가지고 저희가 설정한 설명변수가 품질도에 미치는 영향을 알아보고 싶었습니다. 해당 모델들에서의 변수 중요도를 계산하고 통합하는 과정을 추가로 진행하였습니다. 그 과정과 결과는 아래와 같습니다.

```
# 변수 중요도 저장 리스트 생성
variable_importance <- list()

# 1. 로지스틱 회귀 변수 중요도
log_model <- glm(품질도 ~ 알코올 + 밀도 + 휘발성_산도 + 구연산 + 잔류_당,
  data = train_data, family = binomial)
log_importance <- abs(coef(log_model)) # 계수의 절대값
log_importance <- log_importance[-1] # Intercept 제거
variable_importance$Logistic <- data.frame(Variable = names(log_importance), Importance = log_importance)

# 2. Decision Tree 변수 중요도
library(rpart)
tree_model <- rpart(품질도 ~ 알코올 + 밀도 + 휘발성_산도 + 구연산 + 잔류_당,
  data = train_data, method = "class")
tree_importance <- data.frame(Variable = names(tree_model$variable.importance), Importance = tree_model$variable.importance)
variable_importance$DecisionTree <- tree_importance

# 3. Bagging 변수 중요도
library(randomForest)
bagging_model <- randomForest(품질도 ~ 알코올 + 밀도 + 휘발성_산도 + 구연산 + 잔류_당,
  data = train_data, importance = TRUE)
bagging_importance <- data.frame(Variable = rownames(importance(bagging_model)), Importance = importance(bagging_model)[, 1]) # MeanDecreaseAccuracy
variable_importance$Bagging <- bagging_importance

# 4. Random Forest 변수 중요도
rf_model <- randomForest(품질도 ~ 알코올 + 밀도 + 휘발성_산도 + 구연산 + 잔류_당,
  data = train_data, importance = TRUE)
rf_importance <- data.frame(Variable = rownames(importance(rf_model)), Importance = importance(rf_model)[, 1]) # MeanDecreaseAccuracy
variable_importance$RandomForest <- rf_importance

# 5. SVM 변수 중요도 (선형 커널 가정)
library(e1071)
svm_model <- svm(품질도 ~ 알코올 + 밀도 + 휘발성_산도 + 구연산 + 잔류_당,
  data = train_data, kernel = "linear")
svm_importance <- abs(t(svm_model$coefs) %*% svm_model$SV)
svm_importance_df <- data.frame(Variable = colnames(train_data[, -1]), Importance = as.vector(svm_importance))
variable_importance$SVM <- svm_importance_df

# 6. 변수 중요도 통합
library(dplyr)

# 변수 중요도 테이블 생성
importance_df <- bind_rows(
  Logistic = variable_importance$Logistic,
  DecisionTree = variable_importance$DecisionTree,
  Bagging = variable_importance$Bagging,
  RandomForest = variable_importance$RandomForest,
  SVM = variable_importance$SVM,
  .id = "Model"
)

# 평균 중요도 계산
importance_summary <- importance_df %>%
  group_by(Variable) %>%
  summarise(MeanImportance = mean(Importance, na.rm = TRUE)) %>%
  arrange(desc(MeanImportance))

# 중요도 결과 출력
print(importance_summary)
```



```
# A tibble: 6 × 2
```

Variable	MeanImportance
<chr>	<dbl>
1 알코올	129.
2 휘발성_산도	63.3
3 밀도	47.1
4 잔류_당	35.9
5 구연산	27.5
6 품질도	0.0908

변수 중요도 결과 출력

해당 결과를 보면 품질도의 영향을 미치는 설명변수의 순서는 알코올, 휘발성 산도, 밀도, 잔류당, 구연산 순인 것을 확인해볼 수 있습니다. 이 모델을 이용해 성분들의 함량을 조절해 좋은 품질의 와인을 생산하는 것을 기대합니다.

이 프로젝트를 진행하며 느낀 점은 우선 데이터 분석을 위한 데이터를 찾고 원하는 방향으로 데이터 분석을 완벽히 진행하기가 생각보다 어렵다는 것이었습니다. 특히 데이터를 처음 눈으로 보았을 때와 전처리를 진행하며 자세히 살펴본 데이터는 처음 상상했던 것과는 매우 달랐습니다.

처음에는 소비자를 위한 품질 예측 모델을 만들고자 했으나, 데이터를 분석하는 과정에서 여러 가지 변수가 존재했습니다. 특히 품질이라는 기준이 어떻게 나뉘어졌는지 살펴보았을 때, 이것이 와인 전문가들의 개인적 취향에 따라 결정된다는 것을 알게 되었고, 이는 저희가 모델의 목표와 타겟을 변경하게 된 가장 큰 계기가 되었습니다. 이 과정에서 많은 시행착오를 겪었고, 해결 방안을 찾기 위해 오랜 시간 고민해야 했습니다.

데이터 분석에서 가장 많은 시간이 소요된 부분은 데이터 전처리 과정이었습니다. 또한, 설명변수를 정하는 과정에서도 처음에는 중요도 분석만으로 진행했다가, 프로젝트 마무리 단계에서 상관관계 분석의 필요성을 깨닫고 두 방법을 결합하여 모델링을 다시 진행하게 되었습니다.

최종적인 모델의 성능이 기대했던 것보다 낮아 아쉬움이 남았습니다. 하지만 데이터 분석을 처음 진행하는 만큼 여러 번의 실패와 시행착오는 불가피했다고 생각합니다. 이번 수업과 프로젝트를 통해 데이터를 다루는 것에 익숙해지고 데이터 분석의 기본을 잘 배울 수 있었다는 점에서 큰 보람을 느낍니다.

비록 와인의 품질이 와인 전문가의 주관적인 취향에 영향을 받지만, 저희의 데이터 분석을 통해 도출된 품질 영향 요인들과 각 성분의 함량 기준은 보다 객관적이고 명확한 와인 품질 평가의 기준이 될 수 있다고 생각합니다.

이에 저희는 모델의 예측 목표와 타겟을 성분에 따른 와인의 품질 예측으로 재설정하고 생산자 관점으로 변경하였고, 이 모델을 응용해 실제 와인 생산 과정에서 품질 향상을 위한 가이드라인으로 활용될 수 있기를 기대합니다.